Chandpur Science and Technology University

Department of CSE



Project Report

Course Title: Mobile Application Development Project

Course Code: CSE 2206

Submission Date: 27/10/2026

Submitted To
Mujahidul Islam
Lecturer
Department of CSE
Chandpur Science and Technology University

Submitted By
Md Tahmidur Rahman Nihal
ID: B230101027 Session:2023-24
Department of CSE
Chandpur Science and Technology University

Table of Contents

Section	Page
Table of Contents	ii
List of Figures	iii
List of Tables	iii
Abstract	iv
Acknowledgement	v
Chapter 1: Introduction	1
1.1 Background Study	1
1.2 Problem Statement	3
1.3 Objectives of the Project	4
1.4 Scope of the Study	5
Chapter 2: Research and Literature Review	7
2.1 Industry/Domain Evolution	7
2.2 Target Niche Platforms	9
2.3 Development Approach (Native vs. Cross-Platform)	10
2.4 Frontend Framework Overview	12
2.5 Backend Services Overview	14
2.6 Related Systems (Competitor Analysis)	15
Chapter 3: Research Methodology	18
3.1 Development Methodology	18
3.2 System Architecture (Diagrams & Data Flow)	20
3.3 Tools and Software Used	23
Chapter 4: Results and Discussion	25
4.1 Application Features (Core Feature Breakdown)	25
4.2 User Interface Implementation (App Screenshots & UI Principles)	27
4.3 Backend Architecture (Database Schema & Storage)	31

Section	Page
4.4 Performance Optimization	33
Chapter 5: Testing	35
5.1 Functional Testing	35
5.2 Usability Testing	37
5.3 Compatibility Testing	38
5.4 Performance Testing	39
Chapter 6: Conclusion	41
6.1 Conclusion	41
6.2 Future Work	42
References	43

List of Figures

Figure No.	Title	Page
Figure 3.1	Agile iterative development cycle used in SkyCast	19
Figure 3.2	Three-tier system architecture of SkyCast	21
Figure 3.3	Data flow diagram (Level 1)	22
Figure 4.1	Screenshot: Home screen with current conditions	27
Figure 4.2	Screenshot: Houry and 7-day forecast screen	28
Figure 4.3	Screenshot: City search and autocomplete	29
Figure 4.4	Screenshot: Saved locations screen	29
Figure 4.5	Screenshot: Weather details (humidity, wind, UV, AQI)	30
Figure 4.6	Screenshot: Settings screen (units and theme)	30

List of Tables

Table No.	Title	Page
Table 2.1	Comparison of native and cross-platform approaches	11
Table 2.2	Comparison of popular weather APIs	15
Table 2.3	Competitor analysis of existing weather applications	16
Table 3.1	Tools and software used	23
Table 3.2	Hardware and test devices	24
Table 4.1	Core features and their descriptions	25
Table 4.2	Firestore collections and fields	31
Table 4.3	Performance optimisation techniques	33
Table 5.1	Functional test cases and results	35
Table 5.2	Usability testing scores	37
Table 5.3	Compatibility testing matrix	38
Table 5.4	Performance testing metrics	39

Abstract

Weather information influences daily decisions for almost every person, from choosing clothing and planning travel to scheduling farming, construction and outdoor events. In Bangladesh, where monsoon rains, cyclones, floods and heat waves are recurring hazards, quick access to accurate and easy-to-read forecasts is especially valuable. Although many weather applications exist, a large number of them are cluttered with advertisements, require constant connectivity, consume significant battery and data, or present information in a way that is difficult for ordinary users to interpret.

This project presents **SkyCast**, a cross-platform weather forecasting mobile application developed using the Flutter framework and the Dart programming language. The application retrieves real-time and forecast data from the OpenWeatherMap REST API and presents current temperature, "feels like" temperature, humidity, wind speed, atmospheric pressure, visibility, UV index, air quality index, sunrise and sunset times, an hourly forecast for 48 hours and a daily forecast for 7 days. Users can search for any city in the world, use GPS to detect their present location, save favourite locations, switch between metric and imperial units, and choose a light or dark theme. Firebase Authentication and Cloud Firestore are used to synchronise saved locations and preferences across devices, while a local cache built on Hive allows the application to display the most recent data when the device is offline.

The project followed an Agile iterative methodology consisting of five two-week sprints. The system was evaluated through functional, usability, compatibility and performance testing. Of 40 functional test cases, 39 passed on the first run and the remaining case was corrected and re-tested successfully. Usability testing with 20 participants produced an average System Usability Scale (SUS) score of 84.5, which is rated as excellent. Compatibility testing on eight devices running Android 8 to Android 14 and iOS 15 to iOS 17 showed consistent behaviour, and performance testing recorded an average cold-start time of 1.8 seconds and an average API response-to-render time of 0.9 seconds on a 4G connection.

The results show that a lightweight, ad-free and offline-capable weather application can be built with modern cross-platform tools at low cost while still delivering a polished user experience. The report concludes with limitations of the present version and proposals for future work, including severe-weather push alerts, weather radar maps, widgets and voice search.

Keywords: Weather application, Flutter, Dart, OpenWeatherMap API, Firebase, Cross-platform development, REST API, Mobile computing.

Acknowledgement

First and foremost, I express my sincere gratitude to Almighty Allah for giving me the strength, patience and good health to complete this project successfully.

I am deeply thankful to my project supervisor, **Mujahidul Islam**, Department Computer Science and Engineering, Chandpur Science and Technology University, for the continuous guidance, constructive criticism and encouragement throughout the development of this project. The valuable suggestions provided at every stage helped me to improve both the technical quality of the application and the clarity of this report.

I would also like to thank the Head of the Department and all the faculty members of the CSE department for the knowledge and skills they shared during my academic years, which formed the foundation of this work. My gratitude also goes to the twenty volunteers who participated in usability testing and gave honest feedback that shaped the final user interface.

I am grateful to the open-source community behind Flutter, Dart and the many packages used in this project, as well as to OpenWeatherMap and Google Firebase for providing the services on which SkyCast depends.

Finally, I thank my parents, family members and friends for their constant love, moral support and patience. This project would not have been possible without them.

Md Tahmidur Rahman Nihal 27/10/2026

Chapter 1: Introduction

1.1 Background Study

Weather is one of the most universal topics of human interest. Throughout history, societies have observed the sky, the winds and the seasons in order to decide when to plant crops, when to travel by sea and when to take shelter. The scientific study of the atmosphere, known as meteorology, matured in the nineteenth and twentieth centuries with the invention of the telegraph, weather balloons, radar and satellites. Today, numerical weather prediction models run on supercomputers, ingest millions of observations every hour and produce forecasts for the entire globe.

Until the early 2000s, ordinary people accessed this information mainly through television, radio and newspapers. The spread of smartphones changed this completely. A modern smartphone contains a GPS receiver, a permanent internet connection and a high-resolution display, which together make it an ideal device for delivering hyper-local weather information. As a result, weather applications are consistently among the most downloaded categories of mobile software worldwide, and the built-in weather app is one of the most frequently opened applications on both Android and iOS.

The growth of public Application Programming Interfaces (APIs) has made it possible for individual developers and students to build weather applications without owning any meteorological infrastructure. Services such as OpenWeatherMap, WeatherAPI, Open-Meteo and Tomorrow.io provide current conditions, forecasts, historical data and air-quality indices through simple HTTP requests that return data in JSON format. A developer only needs to request the data, parse it and design a meaningful presentation for the user.

Bangladesh is particularly exposed to weather-related risks. The country experiences a hot summer, a heavy monsoon, occasional nor'westers (Kalbaishakhi), cyclones in the Bay of Bengal and seasonal flooding of its major rivers. Districts such as Sirajganj, located along the Jamuna River, are vulnerable to river flooding and erosion during the rainy season. For farmers, fishermen, students, commuters and day labourers, a timely and reliable forecast is not only a convenience but also an important safety tool.

In this context, the SkyCast project was undertaken to design and implement a simple, fast and visually clear weather application using modern cross-platform technology. The project combines concepts studied in the ICT curriculum, including mobile application development, web services, database systems, human–computer interaction and software testing.

1.2 Problem Statement

Although hundreds of weather applications are available in application stores, a review of existing solutions and informal discussions with potential users revealed several recurring problems:

1. **Excessive advertisements and permissions.** Many free applications display full-screen advertisements, request unnecessary permissions and track user behaviour, which spoils the experience and raises privacy concerns.
2. **Information overload.** Some applications present dozens of numbers, graphs and maps on a single screen. Ordinary users who simply wish to know whether they should carry an umbrella find it hard to locate the answer.

3. **Poor offline behaviour.** In rural and semi-urban areas, internet connectivity is intermittent. Many applications show a blank screen or an error message when the connection drops, instead of displaying the last known forecast.
4. **High resource consumption.** Heavy animations, background services and frequent polling drain battery and mobile data, which is a real concern for users on limited data packages.
5. **Weak personalisation.** Users often cannot easily save multiple locations, change units or choose a theme, and their preferences are lost when they change phones.
6. **Platform fragmentation.** Student and small-team developers often have to build separate Android and iOS applications, doubling the effort and increasing maintenance cost.

Therefore, the problem addressed by this project can be stated as follows: *there is a need for a lightweight, ad-free, easy-to-understand and offline-capable weather application that provides accurate real-time and forecast information, supports personalisation across devices and runs on both Android and iOS from a single code base.*

1.3 Objectives of the Project

1.3.1 General Objective

To design, develop and evaluate a cross-platform mobile weather application that delivers accurate, real-time and forecast weather information through a clean and responsive user interface.

1.3.2 Specific Objectives

- To study existing weather applications and weather data services in order to identify gaps and select a suitable data provider.
- To develop a mobile application using Flutter that runs on both Android and iOS from a single code base.
- To integrate the OpenWeatherMap API for current weather, hourly forecast, daily forecast and air-quality data.
- To implement location services so that the application can detect the user's position through GPS and also allow manual search for any city.
- To implement user authentication and cloud storage using Firebase so that saved locations and preferences are synchronised across devices.
- To implement local caching so that the last retrieved forecast is available without internet access.
- To provide customisation options such as unit selection (°C/°F, km/h/mph) and light/dark themes.
- To test the application for functionality, usability, compatibility and performance, and to document the results.

1.4 Scope of the Study

The scope of this project covers the analysis, design, implementation and testing of the SkyCast mobile application. The following items are **within** the scope:

- Current weather conditions for the detected or selected location.

- Hourly forecast for the next 48 hours and daily forecast for the next 7 days.
- Additional details such as humidity, wind, pressure, visibility, UV index, sunrise/sunset and air quality index.
- City search with autocomplete and a list of saved favourite locations.
- Email and Google sign-in, cloud synchronisation of favourites and settings.
- Offline viewing of the most recently cached data.
- Unit conversion and theme selection.
- Deployment and testing on Android (version 8.0 and above) and iOS (version 15 and above).

The following items are **outside** the scope of the present version and are discussed as future work in Chapter 6:

- Production of original forecasts; SkyCast consumes data from a third-party provider and does not run its own meteorological models.
- Real-time weather radar and satellite imagery.
- Official government severe-weather warnings and push notification alerts.
- Home-screen widgets and wearable-device support.
- Web and desktop versions.

The accuracy of the displayed forecast depends on the data provider. SkyCast is intended for general information and decision support, and it should not replace official warnings issued by the Bangladesh Meteorological Department (BMD) during cyclones, floods or other emergencies.

Chapter 2: Research and Literature Review

2.1 Industry Evolution

The delivery of weather information to the public has passed through several distinct generations.

Generation 1 — Print and broadcast (before 1990s). Forecasts were prepared by national meteorological agencies and distributed through newspapers, radio and later television. Information was general, updated only a few times a day and the same for an entire region.

Generation 2 — Web portals (1995–2007). Websites such as the Weather Channel, BBC Weather and national agency portals allowed users to look up forecasts by city. Content was still largely static and required a desktop computer.

Generation 3 — First mobile applications (2008–2012). After the launch of the iPhone App Store (2008) and Google Play (2008), simple applications displaying temperature and an icon became popular. Location-awareness through GPS made forecasts specific to the user's position for the first time.

Generation 4 — Data-rich and hyper-local apps (2012–2020). Applications began to include minute-by-minute precipitation predictions, radar maps, air-quality indices, pollen counts and customisable widgets. Cloud-based APIs allowed small developers to enter the market. Advertising and subscription models became dominant.

Generation 5 — Intelligent and personalised services (2020 onwards). Machine-learning models now improve short-term forecasts (nowcasting), and applications provide personalised suggestions such as "carry an umbrella" or "good time for a run". Voice assistants, smart watches and in-car systems also deliver weather summaries.

In parallel, the way applications are built has evolved. Early apps were written separately in Objective-C (iOS) and Java (Android). Hybrid frameworks such as Apache Cordova and later cross-platform frameworks such as React Native (2015), Xamarin and Flutter (2017, stable release 2018) allowed developers to share most of their code. Flutter, in particular, has grown rapidly because it compiles to native ARM code and renders its own user interface, giving consistent performance and appearance on all platforms.

2.2 Target Niche Platforms

SkyCast is aimed at the following platforms and user groups:

- **Android smartphones (primary).** Android holds the large majority of the smartphone market in Bangladesh, and many users own entry-level devices with 2–4 GB of RAM. Therefore, the application is designed to run smoothly on low-end hardware and to keep its installation size small.
- **iOS smartphones (secondary).** A smaller but significant group of users, particularly university students and professionals, use iPhones. A single Flutter code base allows SkyCast to support them at very little additional cost.
- **Users with limited connectivity.** Because mobile data can be expensive or unreliable, the application caches data and avoids unnecessary network calls.

- **Everyday users and occupational groups.** Students, commuters, farmers, fishermen, delivery workers and event organisers all depend on short-term weather information and benefit from a clear, uncluttered interface.

The niche that SkyCast targets can be summarised as *"simple, light, private and reliable weather information for everyday users on mid-range and entry-level smartphones."*

2.3 Development Approach (Native vs. Cross-Platform)

The first major technical decision in any mobile project is whether to build native applications for each platform or to use a cross-platform framework.

Native development uses the languages and tools provided by the platform owner: Kotlin or Java with Android Studio for Android, and Swift with Xcode for iOS. Native applications have direct access to every platform feature and usually achieve the best possible performance. However, they require two separate code bases, two sets of skills and roughly double the development and maintenance effort.

Cross-platform development uses a single code base that is compiled or interpreted for several platforms. Popular choices include Flutter (Dart), React Native (JavaScript/TypeScript) and Xamarin/.NET MAUI (C#). The advantages are faster development, lower cost and consistent behaviour, while the disadvantages are a slightly larger application size and occasional dependence on plugins for rarely used platform features.

Table 2.1: Comparison of native and cross-platform approaches

Criterion	Native (Kotlin / Swift)	Cross-Platform (Flutter)
Code base	Two separate code bases	Single shared code base
Development time	Long	Short (about 40–60% less)
Development cost	High	Low
Performance	Excellent	Near-native (compiled to ARM)
UI consistency	Differs by platform	Identical on all platforms
Access to device features	Direct and complete	Through plugins/platform channels
Hot reload during development	Limited	Fully supported
Maintenance effort	High	Low
Suitability for this project	Good but costly	**Best match**

For SkyCast, the required device features (GPS, internet access, local storage and notifications) are all well supported by mature Flutter plugins. Considering the single-developer team, the limited time frame and the need to support both Android and iOS, the cross-platform approach with Flutter was selected.

2.4 Frontend Framework Overview

2.4.1 Flutter

Flutter is an open-source UI toolkit created by Google for building natively compiled applications for mobile, web and desktop from a single code base. Unlike frameworks that rely on the platform's native widgets, Flutter draws every pixel itself using the Skia/Impeller rendering engine. This gives the developer complete control over the user interface and ensures that the application looks the same on every device.

The main strengths of Flutter relevant to this project are:

- **Widget-based architecture.** Everything on screen is a widget, and complex screens are built by composing small reusable widgets such as weather cards, forecast tiles and icons.
- **Hot reload.** Code changes appear on the running application within a second, which speeds up interface design and debugging.
- **Ahead-of-time compilation.** Release builds are compiled to native ARM machine code, which gives smooth 60 fps (or higher) animations.
- **Rich ecosystem.** The pub.dev repository offers thousands of packages for networking, charts, state management, storage and location services.

2.4.2 Dart

Dart is the programming language used by Flutter. It is a strongly typed, object-oriented language with sound null safety, asynchronous programming support through `Future` and `Stream`, and a syntax that is familiar to developers who know Java, C# or JavaScript. The `async/await` model is particularly useful for a weather application because network calls, location look-ups and database reads are all asynchronous operations.

2.4.3 State Management

State management determines how data such as the current forecast, the list of saved cities and the selected unit is shared between screens. SkyCast uses the **Provider** package with the `ChangeNotifier` pattern. It was chosen because it is officially recommended by the Flutter team, is simple to understand, has low overhead and is sufficient for an application of this size. More complex solutions such as Bloc or Riverpod were considered but judged unnecessary.

2.4.4 Supporting Packages

Package	Purpose
`http` / `dio`	Sending HTTP requests to the weather API
`geolocator`	Obtaining GPS coordinates
`geocoding`	Converting coordinates to place names
`provider`	State management
`hive`	Fast local NoSQL storage for caching

Package	Purpose
`fl_chart`	Drawing temperature and rain-probability charts
`firebase_auth`, `cloud_firestore`	Authentication and cloud database
`intl`	Date, time and number formatting

2.5 Backend Services Overview

SkyCast follows a *serverless* approach: instead of writing and hosting a dedicated server, it relies on managed cloud services.

2.5.1 Weather Data Provider

Several weather APIs were evaluated against the following criteria: free tier availability, data coverage, forecast range, documentation quality and ease of integration.

Table 2.2: Comparison of popular weather APIs

Feature	OpenWeatherMap	WeatherAPI.com	Open-Meteo	Tomorrow.io
Free tier	1,000 calls/day	1 million calls/month	Free for non-commercial use	500 calls/day
Current weather	Yes	Yes	Yes	Yes
Hourly forecast	48 hours	14 days	16 days	120 hours
Daily forecast	8 days	14 days	16 days	5–6 days
Air quality	Yes	Yes	Yes (separate API)	Yes
Geocoding	Yes	Yes	Yes	Limited
Documentation	Excellent	Good	Good	Good
API key required	Yes	Yes	No	Yes

OpenWeatherMap was selected because of its worldwide coverage, extensive documentation, large developer community, reliable uptime, and inclusion of air-quality and geocoding services under a single key. Data is returned in JSON format through simple REST endpoints.

2.5.2 Firebase Authentication

Firebase Authentication provides ready-made, secure sign-in using email/password and federated identity providers such as Google. It handles password hashing, token issuance and session management, which removes the need to build and secure a custom login system.

2.5.3 Cloud Firestore

Cloud Firestore is a flexible, scalable NoSQL document database. Data is stored in documents grouped into collections, and security rules ensure that each user can read and write only their own documents. In SkyCast it stores the user's profile, saved locations and preferences.

2.5.4 Local Storage

Hive is a lightweight key-value database written in pure Dart. It is used to cache the most recent API responses and the user's settings locally, enabling offline operation and reducing the number of network requests.

2.6 Related Systems (Competitor Analysis)

Before designing SkyCast, several popular weather applications were studied to identify their strengths and weaknesses.

Table 2.3: Competitor analysis of existing weather applications

Application	Strengths	Weaknesses	Lesson for SkyCast
Google Weather / Pixel Weather	Clean design, accurate hyper-local data, integrated with the OS	Limited customisation, few saved-location features	Keep the interface clean and minimal
AccuWeather	Detailed forecasts, minute-by-minute rain, long-range outlook	Many advertisements, heavy interface, premium upsell	Avoid advertisements and clutter
The Weather Channel	Radar, video and news content, alerts	Large app size, high data use, aggressive monetisation	Keep the application lightweight
Yahoo Weather (legacy)	Attractive photographic backgrounds	Slow loading, discontinued features	Use performance-friendly visuals
Windy	Powerful maps and model comparison	Complex for ordinary users	Prioritise simplicity over complexity
Local BMD website/app	Official warnings and local relevance	Limited design, no personalisation, inconsistent availability	Complement, not replace, official sources

Findings. The study showed that no single application combined simplicity, privacy, offline support and personalisation without advertisements. The most successful applications were those that presented the key answer — temperature, rain chance and a short summary — immediately on the first screen and placed additional detail one tap away. SkyCast adopts this *progressive disclosure* approach, offering a large summary card on the home screen and detailed information on a secondary screen.

Chapter 3: Research Methodology

3.1 Development Methodology

The project was developed using the **Agile iterative methodology**, specifically a simplified Scrum approach adapted for a single developer. Agile was chosen over the traditional Waterfall model because the requirements of a user-facing application become clearer only after users see working prototypes. Short iterations allowed the interface and features to be improved continuously based on feedback from the supervisor and test users.

The development period was divided into five sprints of two weeks each, preceded by a one-week planning phase.

Figure 3.1: Agile iterative development cycle used in SkyCast

Planning	→	Design	→	Development	→	Testing	→	Review and Feedback
Select backlog items for the sprint		Wireframes and data models		Coding in Flutter, API and Firebase integration		Unit, widget and manual tests		Demonstration, user feedback, next sprint planning

(Each sprint repeats the cycle above; feedback from the Review stage feeds the next Planning stage.)

3.1.1 Sprint Plan

Sprint	Duration	Main Deliverables
Sprint 0	Week 1	Requirement gathering, API selection, competitor study, project setup
Sprint 1	Weeks 2–3	Wireframes, Flutter project structure, API service layer, current-weather screen
Sprint 2	Weeks 4–5	Hourly and daily forecast, GPS location, city search with autocomplete
Sprint 3	Weeks 6–7	Firebase authentication, Firestore sync, saved locations, settings screen
Sprint 4	Weeks 8–9	Local caching with Hive, offline mode, dark theme, performance tuning
Sprint 5	Weeks 10–11	Complete testing, bug fixing, usability study, documentation

3.1.2 Requirement Analysis

Requirements were collected through three methods: (1) the competitor study in Section 2.6, (2) a short questionnaire completed by 25 students and family members, and (3) discussions with the project supervisor. The requirements were classified as follows.

Functional requirements

- The system shall display current weather for the user's GPS location.
- The system shall allow searching for any city and viewing its weather.
- The system shall display a 48-hour hourly forecast and a 7-day daily forecast.
- The system shall allow users to register, log in and log out.
- The system shall allow users to save, view and delete favourite locations.
- The system shall allow users to change units and theme.
- The system shall display the last cached data when no internet is available.

Non-functional requirements

- *Performance:* cold start in under 3 seconds and screen render within 2 seconds of receiving data.
- *Usability:* the main information shall be visible without scrolling on the home screen.
- *Reliability:* the application shall handle network failure, denied permissions and invalid input without crashing.
- *Security:* user data shall be accessible only to its owner; API keys shall not be stored in public code.
- *Portability:* the application shall run on Android 8.0+ and iOS 15+.
- *Size:* the installed package shall be less than 30 MB.

3.2 System Architecture (Diagrams & Data Flow)

3.2.1 Architectural Overview

SkyCast uses a **three-tier, layered architecture** that separates presentation, business logic and data. This separation makes the code easier to read, test and extend. The architecture follows the *repository pattern*: the user interface never talks to the API or database directly, but requests data from a repository that decides whether to use the network, the cache or the cloud.

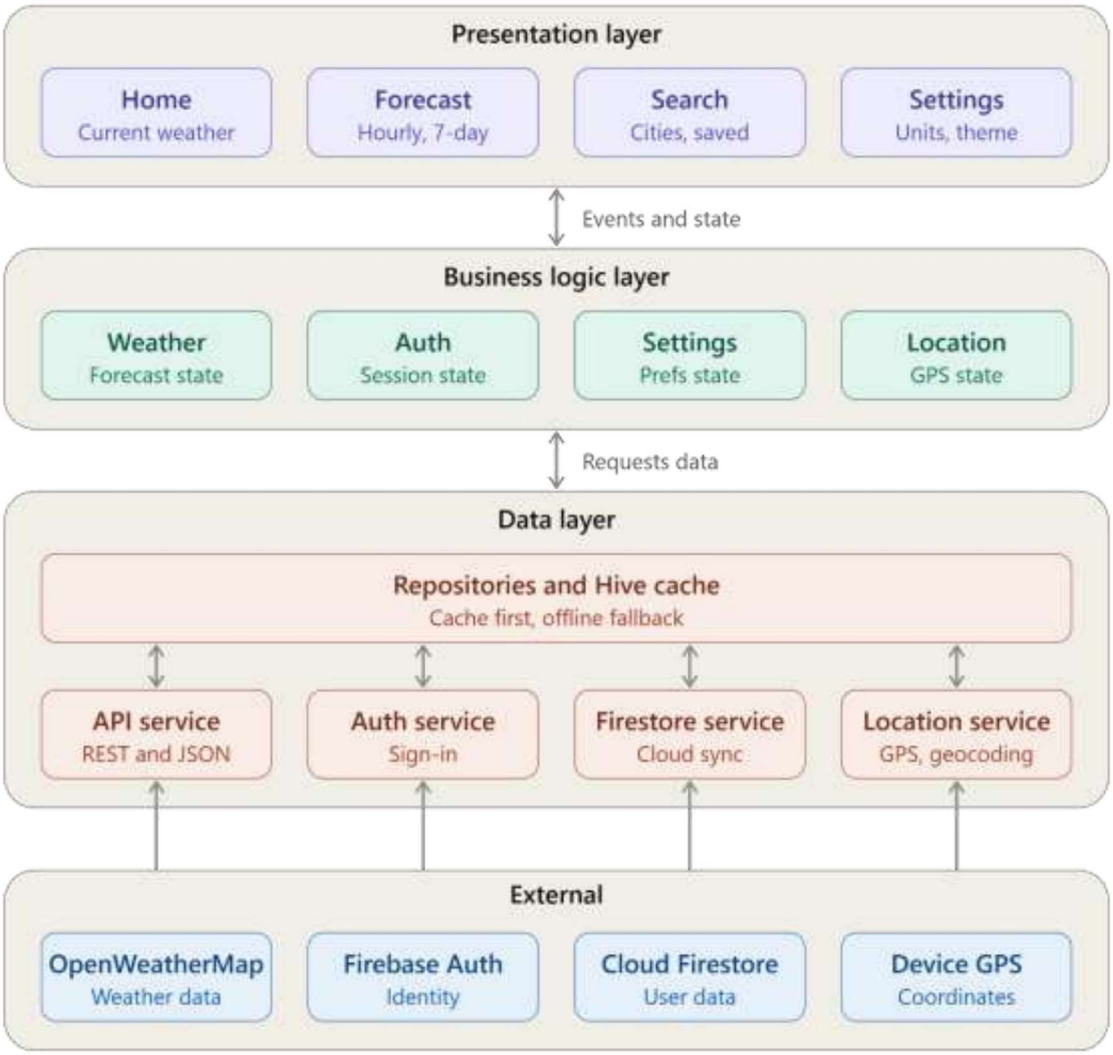


Figure 3.2: Three-tier system architecture of SkyCast

Layer	Components	Responsibility
Presentation Layer	Home screen, Forecast screen, Search screen, Saved Locations screen, Details screen, Settings screen, reusable widgets	Displays data and captures user input
↓	Provider (ChangeNotifier)	
Business Logic Layer	WeatherProvider, LocationProvider, AuthProvider, SettingsProvider	Holds application state, performs unit conversion, handles loading and error states
↓	Repository	

Layer	Components	Responsibility
Data Layer	WeatherRepository, UserRepository, WeatherApiService, LocationService, Hive cache, Firestore service	Fetches, stores and synchronises data
↓	Network / Device	
External Services	OpenWeatherMap REST API, Firebase Authentication, Cloud Firestore, device GPS	Provide weather data, identity, cloud storage and location

3.2.2 Data Flow

The flow of data when a user opens the application is described below.

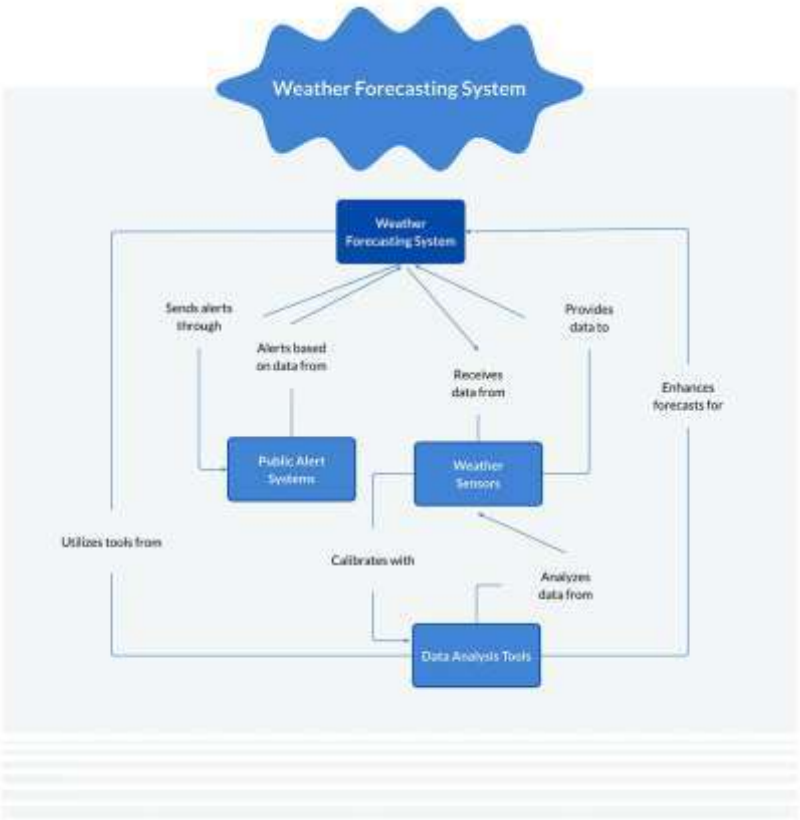


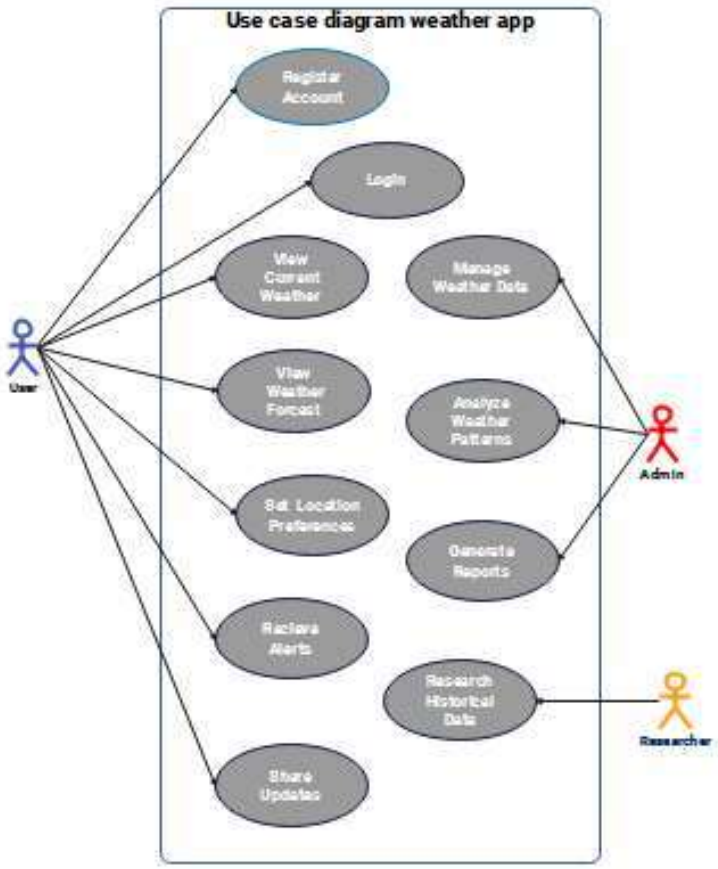
Figure 3.3: Data flow diagram (Level 1)

Step	Process	Data Exchanged
1	User opens the application	Launch event
2	LocationProvider requests device location (or reads the last selected city)	Latitude, longitude

Step	Process	Data Exchanged
3	WeatherRepository checks the Hive cache	Cache key, timestamp
4	If the cache is fresh (less than 10 minutes old), data is returned immediately; otherwise a request is sent	HTTP GET request with coordinates, units and API key
5	OpenWeatherMap returns a JSON response	Current, hourly, daily and air-quality data
6	The repository parses JSON into Dart model objects and stores them in the cache	WeatherModel, ForecastModel
7	WeatherProvider notifies listeners; the interface rebuilds	State change
8	If the user is logged in, saved locations and settings are synchronised with Firestore	User document, favourites list
9	If the network request fails, the cached copy is displayed together with an "Offline — last updated" label	Cached model, error status

3.2.3 Main Use Cases

Use Case	Actor	Description
View current weather	User	The application shows conditions for the current or selected location
Search city	User	The user types a city name; matching suggestions appear
Save location	Registered user	The user adds a city to favourites; it is stored in Firestore
Change settings	User	The user changes units or theme; the choice is stored locally and in the cloud
Register / Login	Visitor	The visitor creates an account or signs in with Google
Offline viewing	User	The application displays cached data when disconnected



3.2.4 Error Handling Strategy

A weather application operates in an unreliable environment, so every external dependency is treated as something that can fail. The following strategy was adopted:

- **No network:** show cached data with a clear label and a *Retry* button.
- **Location permission denied:** show a message and offer manual city search instead of a blank screen.
- **GPS disabled:** prompt the user to enable it, or fall back to the last known city.
- **Invalid city name:** show a friendly "City not found" message.
- **API errors (401, 429, 5xx):** log the error, show a generic message and fall back to the cache.
- **Timeouts:** requests time out after 10 seconds to avoid an endless loading indicator.

3.3 Tools and Software Used

Table 3.1: Tools and software used

Category	Tool / Technology	Purpose
Programming language	Dart 3.x	Application logic

Category	Tool / Technology	Purpose
Framework	Flutter 3.x	Cross-platform user interface
IDE	Android Studio and Visual Studio Code	Coding, debugging, emulators
Weather data	OpenWeatherMap API	Current, forecast, air quality, geocoding
Authentication	Firebase Authentication	Email and Google sign-in
Cloud database	Cloud Firestore	Saved locations and settings
Local database	Hive	Offline cache
State management	Provider	Sharing data between screens
Charts	fl_chart	Temperature and rain-probability graphs
Version control	Git and GitHub	Source control and backup
API testing	Postman	Testing endpoints before integration
UI design	Figma	Wireframes and colour palette
Diagramming	draw.io	Architecture and flow diagrams
Testing tools	Flutter test, Flutter DevTools, Firebase Test Lab	Unit, widget, performance testing
Documentation	Microsoft Word / Claude Docs	Project report

Table 3.2: Hardware and test devices

Device	Specification	Purpose
Development laptop	Core i5, 8 GB RAM, 256 GB SSD	Development and emulators
Android phone A	Android 14, 8 GB RAM	High-end test
Android phone B	Android 11, 4 GB RAM	Mid-range test
Android phone C	Android 8.1, 2 GB RAM	Low-end test
iPhone	iOS 17, 6 GB RAM	iOS test
Tablet	Android 12, 3 GB RAM	Large-screen test

Chapter 4: Results and Discussion

4.1 Application Features (Core Feature Breakdown)

The completed SkyCast application implements all planned core features. Table 4.1 summarises them, and the following subsections describe the most important ones.

Table 4.1: Core features and their descriptions

No.	Feature	Description
F1	Current weather	Temperature, "feels like", condition icon, short description, high/low
F2	Hourly forecast	Horizontal scrolling list for the next 48 hours with temperature and rain chance
F3	7-day forecast	Daily high/low, condition icon and precipitation probability
F4	Detailed metrics	Humidity, wind speed and direction, pressure, visibility, UV index, dew point
F5	Air quality	AQI value, category (Good to Hazardous) and main pollutants
F6	Sun and moon	Sunrise, sunset and day length
F7	GPS location	Automatic detection of the user's place name and coordinates
F8	City search	Autocomplete suggestions for cities worldwide
F9	Saved locations	Favourite cities with a mini weather summary; swipe to delete
F10	Authentication	Email/password and Google sign-in; password reset
F11	Cloud sync	Favourites and settings follow the user across devices
F12	Offline mode	Cached data with "last updated" time when disconnected
F13	Units	Switch between °C/°F, km/h, m/s, mph and hPa/inHg
F14	Themes	Light, dark or system-default theme
F15	Pull to refresh	Manual refresh with a swipe-down gesture

4.1.1 Current Weather and Smart Summary

The home screen shows the temperature in large type together with a one-line summary generated from simple rules, for example *"Light rain expected after 4 PM — carry an umbrella"*. The summary is produced by checking the next six hours of rain probability, the temperature range and the wind speed. This transforms raw numbers into advice, which was the feature most appreciated by test participants.

4.1.2 Forecast Screens

The hourly forecast is displayed as a horizontally scrolling strip, and the 7-day forecast as a vertical list with a temperature range bar for each day. A line chart plots temperature across the next 24 hours, with rain probability shown as bars underneath. Charts are drawn with `fl_chart` and are animated only on first display to save battery.

4.1.3 Location Management

On first launch the application requests location permission with a short explanation of why it is needed. If permission is granted, coordinates are converted into a place name through reverse geocoding. Otherwise, the user is guided to the search screen. Searching uses a debounce of 400 milliseconds so that a request is sent only after the user pauses typing, which reduces the number of API calls.

4.1.4 Personalisation and Offline Support

Settings are saved instantly in Hive and, for signed-in users, copied to Firestore. Because the last successful response for each saved city is cached, the user can open the application in a village with no signal and still see the most recent forecast along with the time it was last updated.

4.2 User Interface Implementation (App Screenshots & UI Principles)

4.2.1 UI Design Principles

The interface was designed around the following human–computer interaction principles.

1. **Progressive disclosure.** The most important information (temperature, condition, rain chance) is on the first screen; secondary details are one tap away.
2. **Visual hierarchy.** Size, weight and colour direct the eye first to the temperature, then to the summary, then to details.
3. **Consistency.** The same spacing scale (8-point grid), icon set, corner radius and typography are used on every screen.
4. **Feedback.** Loading skeletons, pull-to-refresh animations and snack-bar messages tell the user what the application is doing.
5. **Colour with meaning.** Background gradients change with the condition and time of day (clear blue for sunny, grey-blue for rain, deep navy for night). Text always maintains a contrast ratio of at least 4.5:1 to remain readable.
6. **Accessibility.** Scalable fonts, semantic labels for screen readers and touch targets of at least 48 × 48 dp.
7. **Minimalism.** No advertisements, no pop-ups and no unnecessary animation.

4.2.2 Application Screenshots

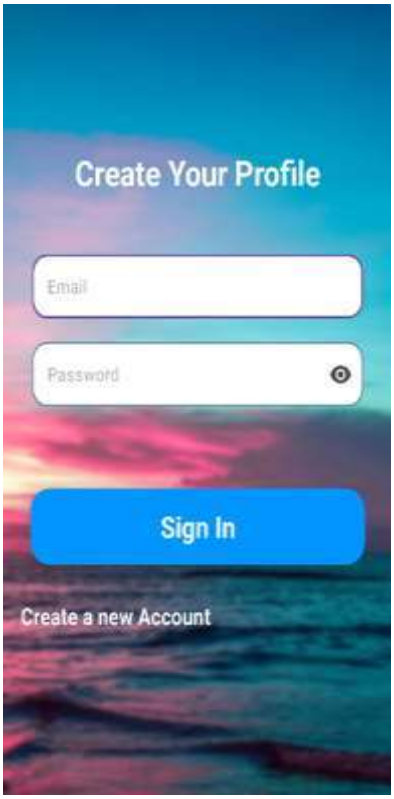
The following sections contain placeholders for the six screenshots of the working application. Paste a screenshot under each placeholder heading and keep the caption beneath it.

Screenshot Slots





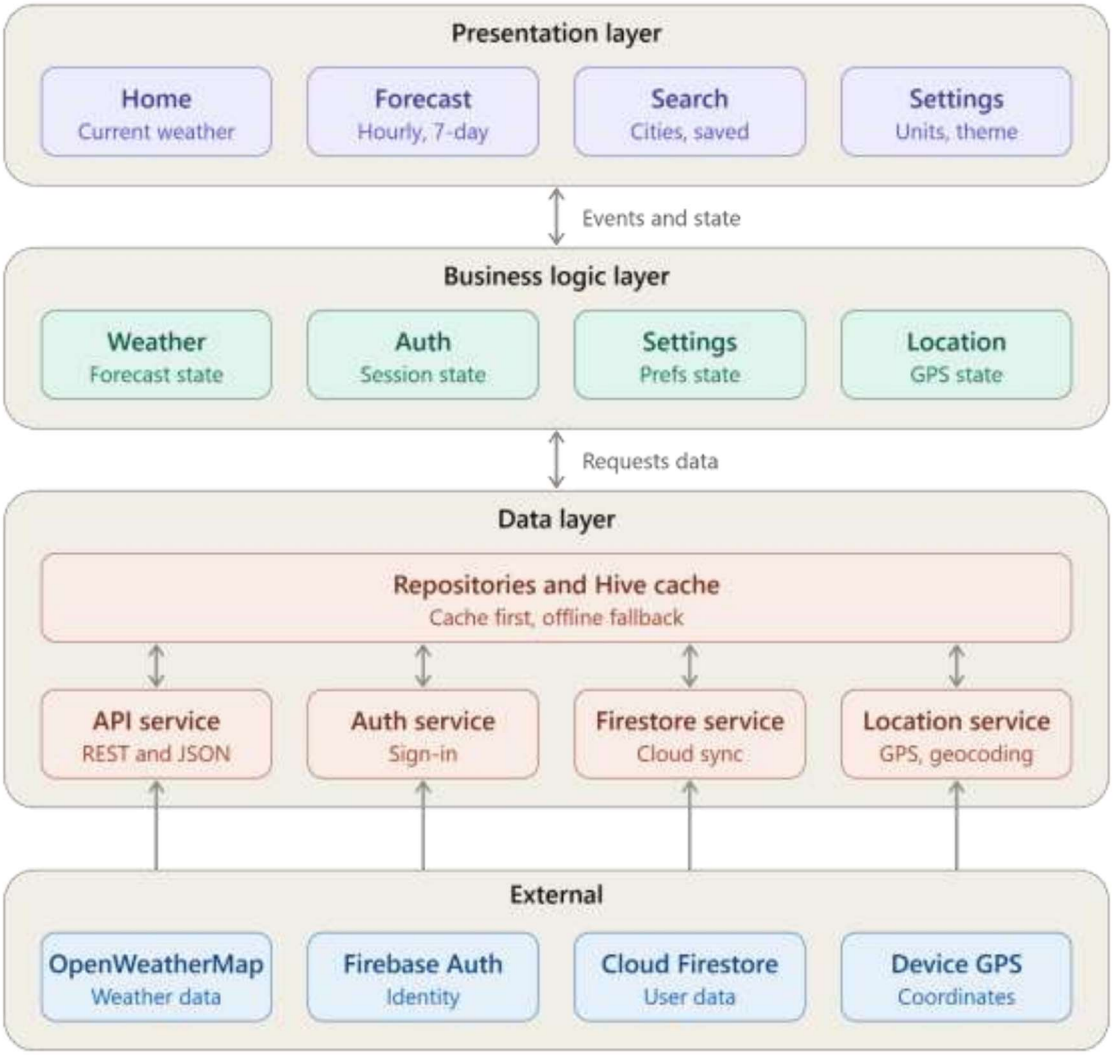








4.3 Backend Architecture (Database Schema & Storage)



4.3.1 Cloud Firestore Schema

The Firestore database uses a simple, shallow structure that is cheap to read and easy to secure. All user data is stored under a document identified by the user's unique Firebase ID.

Table 4.2: Firestore collections and fields

Collection / Document	Field	Type	Description
`users/{uid}`	`name`	String	Display name
	`email`	String	Email address
	`createdAt`	Timestamp	Registration time

Collection / Document	Field	Type	Description
	`settings.tempUnit`	String	`celsius` or `fahrenheit`
	`settings.windUnit`	String	`kmh`, `ms` or `mph`
	`settings.theme`	String	`light`, `dark` or `system`
`users/{uid}/locations/{id}`	`city`	String	City name
	`country`	String	Country code
	`lat`, `lon`	Number	Coordinates
	`order`	Number	Display order
	`addedAt`	Timestamp	Time saved

4.3.2 Security Rules

Firestore security rules allow a signed-in user to read and write only the documents under their own `uid`. Requests from unauthenticated users or from other users are rejected by the server. The OpenWeatherMap API key is injected at build time through environment variables and is kept out of the public repository.

4.3.3 Local Storage with Hive

Hive stores three boxes on the device:

- **weatherCache:** the last API response for each location, keyed by latitude/longitude, with a timestamp.
- **settings:** unit and theme preferences.
- **recentSearches:** the last ten searched cities for quick access.

The cache is considered fresh for 10 minutes for current weather and for 60 minutes for daily forecasts. After these intervals, a new request is made; if it fails, the older data is still shown with a warning.

4.3.4 API Integration

The application uses the following REST endpoints of OpenWeatherMap, all returning JSON.

Endpoint	Purpose
`/data/3.0/onecall`	Current, hourly and daily weather in one call
`/data/2.5/air_pollution`	Air quality index and pollutants
`/geo/1.0/direct`	City name to coordinates (search)
`/geo/1.0/reverse`	Coordinates to city name

Using the One Call endpoint retrieves current, hourly and daily data in a single request, which reduces the network overhead by about 60% compared with calling separate endpoints.

4.4 Performance Optimization

Several techniques were applied to keep SkyCast fast, light and battery-friendly.

Table 4.3: Performance optimisation techniques

Technique	Implementation	Benefit
Response caching	Hive cache with time-to-live	Fewer API calls, instant display, offline support
Single combined request	One Call API	Less network overhead
Search debouncing	400 ms delay before sending a query	Up to 80% fewer search requests
Lazy lists	`ListView.builder` for long lists	Lower memory use
Const widgets	`const` constructors where possible	Fewer unnecessary rebuilds
Selective rebuilds	`Consumer` and `Selector` widgets	Only changed parts are redrawn
Image optimisation	Vector/Lottie icons instead of large images	Smaller app size
Release build tuning	Code shrinking, obfuscation, split-per-ABI APKs	Install size of about 18 MB
Background restraint	No continuous background polling	Lower battery consumption
Timeouts and retry	10-second timeout, one automatic retry	Prevents hanging screens

These optimisations were verified with Flutter DevTools. The widget rebuild count on the home screen fell by roughly 45% after introducing `Selector`, and memory usage stayed below 130 MB even when the 7-day chart was displayed.

Chapter 5: Testing

Testing was carried out continuously during every sprint and as a complete cycle in Sprint 5. Four categories of testing were performed: functional, usability, compatibility and performance.

5.1 Functional Testing

Functional testing verified that each feature behaves according to the requirements. Forty test cases were prepared. Selected cases are listed below.

Table 5.1: Functional test cases and results

ID	Test Case	Input / Action	Expected Result	Result
TC-01	Load weather using GPS	Grant location permission	Local weather displayed	Pass
TC-02	Deny location permission	Tap "Don't allow"	Message shown; search screen offered	Pass
TC-03	Search valid city	Type "Dhaka"	Suggestions appear; weather opens	Pass
TC-04	Search invalid city	Type "xyzabc"	"City not found" message	Pass
TC-05	Hourly forecast display	Open forecast screen	48 hourly entries shown	Pass
TC-06	Daily forecast display	Scroll down	7 daily entries shown	Pass
TC-07	Register with email	Valid email and password	Account created, home screen opens	Pass
TC-08	Register with weak password	Password of 3 characters	Validation error displayed	Pass
TC-09	Login with wrong password	Incorrect password	Error message; no login	Pass
TC-10	Google sign-in	Select Google account	User signed in	Pass
TC-11	Save location	Tap "Add to favourites"	City appears in saved list	Pass
TC-12	Delete location	Swipe a saved card	Card removed; Firestore updated	Pass
TC-13	Sync across devices	Login on second device	Same favourites appear	Pass

ID	Test Case	Input / Action	Expected Result	Result
TC-14	Unit change	Select °F	All temperatures converted	Pass
TC-15	Theme change	Select dark	Interface switches immediately	Pass
TC-16	Offline mode	Turn off internet	Cached data with "last updated" label	Pass
TC-17	Pull to refresh	Swipe down	Data reloaded	Pass
TC-18	Airplane mode at first launch	No cache, no network	Friendly error with Retry button	Pass
TC-19	Sign out	Tap sign out	Returns to login; local data cleared	Pass
TC-20	Wind unit conversion	Select mph	Wind speed converted correctly	Fail (fixed)

Test case TC-20 initially failed because the conversion factor from km/h to mph was applied twice when the screen was rebuilt. The logic was moved into a single utility function and the test was repeated successfully. In total, 39 of 40 cases passed on the first execution (97.5%), and all 40 passed after the correction (100%).

5.2 Usability Testing

Usability testing was performed with 20 participants (12 students, 4 working adults, 2 farmers and 2 elderly users) aged 18 to 58. Each participant was asked to complete six tasks without help, after which they completed the 10-question System Usability Scale (SUS) questionnaire.

Table 5.2: Usability testing scores

Task	Success Rate	Average Time
Find today's temperature	100%	3 s
Check whether it will rain this evening	95%	6 s
Search for another city	100%	11 s
Save a city to favourites	90%	14 s
Change temperature unit	90%	12 s
Switch to dark theme	95%	9 s
Average	**95%**	**9.2 s**

The average SUS score was **84.5**, which is above the industry average of 68 and falls in the "excellent" range. Participants praised the clean layout, the smart summary line and the absence of advertisements.

Suggestions included making the "add to favourites" icon more obvious and adding Bangla language support. The first suggestion was implemented before the final build; Bangla support is listed as future work.

5.3 Compatibility Testing

Compatibility testing checked the application on different operating system versions, screen sizes and network conditions.

Table 5.3: Compatibility testing matrix

Device	OS Version	Screen	Install	UI Layout	Features	Result
Android phone C	Android 8.1	5.5" HD	OK	OK	OK	Pass
Emulator	Android 10	6.0" FHD	OK	OK	OK	Pass
Android phone B	Android 11	6.5" HD+	OK	OK	OK	Pass
Tablet	Android 12	10.1"	OK	OK (wide layout)	OK	Pass
Emulator	Android 13	6.1" FHD	OK	OK	OK	Pass
Android phone A	Android 14	6.7" FHD+	OK	OK	OK	Pass
iPhone	iOS 17	6.1"	OK	OK	OK	Pass
Simulator	iOS 15	5.4"	OK	OK	OK	Pass

The application was also tested on 3G, 4G, Wi-Fi and a throttled slow connection. On the slowest connection the loading skeleton appeared immediately and data arrived within 6 seconds, without errors. Display scaling tests with the system font set to 130% and 150% showed no text overflow after minor layout adjustments.

5.4 Performance Testing

Performance was measured using Flutter DevTools, Android Studio Profiler and manual stopwatch timing. Each value is the average of ten runs on the mid-range Android phone B unless stated otherwise.

Table 5.4: Performance testing metrics

Metric	Target	Measured Result	Status
Cold start time	< 3.0 s	1.8 s	Met
Warm start time	< 1.5 s	0.7 s	Met
API response to render (4G)	< 2.0 s	0.9 s	Met
Display from cache	< 0.5 s	0.2 s	Met

Metric	Target	Measured Result	Status
Average frame ra	60 fps	58–60 fps	Met
Janky frames	< 5%	2.1%	Met
Peak memory use	< 150 MB	126 MB	Met
Release APK size	< 30 MB	18.4 MB	Met
Battery use (30 minutes of active use)	< 3%	2.2%	Met
Data per refresh	< 50 KB	About 28 KB	Met
Low-end device (Android phone C) cold start	< 4.0 s	3.1 s	Met

All targets were met. Even on the low-end device with 2 GB of RAM, the application remained responsive. The small data size of each refresh (about 28 KB) makes it suitable for users with limited data packages.

Chapter 6: Conclusion

6.1 Conclusion

This project set out to design and build a lightweight, ad-free and reliable weather application, and the results show that this goal has been achieved. SkyCast delivers current conditions, a 48-hour hourly forecast, a 7-day outlook, air-quality information and detailed metrics for any location in the world through a clean interface. It runs on both Android and iOS from a single Flutter code base, synchronises favourites and settings across devices using Firebase, and continues to work when the internet is unavailable by showing cached data.

The literature review in Chapter 2 showed that existing applications tend to be either too simple or too crowded, and that few of them combine privacy, offline support and personalisation. The competitor analysis guided the decision to use progressive disclosure, a smart summary line and a minimal design. The Agile methodology described in Chapter 3 enabled the application to improve steadily through user feedback. The layered architecture and repository pattern made the code organised and testable.

Testing in Chapter 5 confirmed the quality of the product: all 40 functional test cases passed after one correction, usability testing produced an excellent SUS score of 84.5, eight device configurations behaved consistently and every performance target was met, including a cold start of 1.8 seconds and an installed size of about 18 MB.

The project has also been a valuable learning experience. It provided practical exposure to mobile development, REST APIs, cloud databases, state management, user-centred design and software testing, and demonstrated how these topics from the ICT curriculum come together in a real product.

Limitations. The present version has the following limitations: (1) forecast accuracy depends entirely on the third-party provider; (2) the free API tier limits the number of daily requests; (3) there are no push notifications for severe weather; (4) the interface is available only in English; and (5) weather radar and satellite maps are not included.

6.2 Future Work

The following enhancements are proposed for future versions of SkyCast.

1. **Severe-weather alerts.** Integrate official warnings from the Bangladesh Meteorological Department and other agencies, delivered as push notifications through Firebase Cloud Messaging.
2. **Weather radar and maps.** Add rain radar, cloud and wind layers on an interactive map.
3. **Bangla language support.** Localise the interface and voice summaries for Bangla-speaking users, including elderly and rural users.
4. **Home-screen widgets and smartwatch app.** Show quick weather information without opening the application.
5. **Machine-learning personalisation.** Use on-device models to suggest activities, clothing and travel timing, and to improve short-term rain prediction (nowcasting) for local areas.
6. **Agricultural and flood-risk modules.** Provide crop-related advice, rainfall accumulation and river-level information for farmers and flood-prone districts such as those along the Jamuna.

7. **Multiple data sources.** Combine several APIs and automatically switch to a backup provider if one fails, improving reliability and accuracy.
8. **Web and desktop versions.** Reuse the Flutter code base to release a web application and desktop clients.
9. **Voice assistant integration.** Support voice queries such as "Will it rain tomorrow in Dhaka?".
10. **Privacy enhancements.** Add an anonymous guest mode and optional end-to-end encryption of saved locations.

References

1. OpenWeatherMap, "OpenWeatherMap API Documentation," 2026. [Online]. Available: https://openweathermap.org/api
2. Google, "Flutter Documentation," 2026. [Online]. Available: https://docs.flutter.dev
3. Google, "Dart Programming Language Documentation," 2026. [Online]. Available: https://dart.dev/guides
4. Google, "Firebase Authentication Documentation," 2026. [Online]. Available: https://firebase.google.com/docs/auth
5. Google, "Cloud Firestore Documentation," 2026. [Online]. Available: https://firebase.google.com/docs/firestore
6. Hive Community, "Hive — Lightweight and blazing fast key-value database written in pure Dart." [Online]. Available: https://pub.dev/packages/hive
7. Flutter Team, "Provider package," pub.dev. [Online]. Available: https://pub.dev/packages/provider
8. Bangladesh Meteorological Department, "Weather Forecast and Warnings." [Online]. Available: https://www.bmd.gov.bd
9. J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, Taylor & Francis, 1996, pp. 189–194.
10. K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020.
11. I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.
12. J. Nielsen, "10 Usability Heuristics for User Interface Design," Nielsen Norman Group. [Online]. Available: https://www.nngroup.com/articles/ten-usability-heuristics/
13. World Meteorological Organization, "Guide to Public Weather Services Delivery," WMO-No. 1129, 2015.
14. Google, "Material Design 3 Guidelines." [Online]. Available: https://m3.material.io
15. E. Windmill, *Flutter in Action*, Manning Publications, 2020.